

Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models
Student Name	S.S.Janane
Register Number	312247001024
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	01/08/26

1 Objective

The objective of this experiment is to compare four regression models: Linear Regression, Ridge, Lasso, and Elastic Net. The models are evaluated based on how well they predict the sanctioned loan amount. Another goal is to understand how regularization changes the learned coefficients and whether it improves performance on unseen data.

2 Problem Statement

Given a set of loan applicant attributes , income, co-applicant income, requested loan term, credit history, and a handful of demographic/categorical fields , the task is to predict LoanAmount, a continuous variable representing the loan amount sanctioned (in thousands). This is a supervised regression problem. The input is a mix of numerical and categorical features with a non-trivial amount of missing data; the output is a single real number per applicant. The end goal is not just to produce a working model but to understand how L1 and L2 regularization reshape the coefficients learned from this particular dataset, and whether that reshaping actually translates into better generalization.

3 Dataset Description

Dataset Name	Loan Prediction Problem Dataset (train-loan.csv)
Dataset Source	Kaggle - Loan Prediction Problem Dataset
Number of Samples	614 total; 592 remain after dropping rows with a missing target
Number of Features	11 raw predictor columns (excluding Loan_ID and Loan_Status); 4 numerical + 6 categorical, expanding to 19 columns after one-hot encoding
Number of Classes	Not applicable; the target is continuous
Missing Values	7 of the 13 original columns contain missing values: Credit_History (50), Self_Employed (32), LoanAmount (22), Dependents (15), Loan_Amount_Term (14), Gender (13), Married (3)

Train-Test Split	80:20, giving 473 training rows and 119 test rows (random_state=42)
------------------	---

The 592 usable rows have four numerical predictors -ApplicantIncome, CoapplicantIncom, Loan_Amount_Term, Credit_History - and six categorical predictors - Gender, Married, Dependents, Education, Property_Area. The target, LoanAmount, has a mean of 146.41 and a standard deviation of 85.59, with values ranging from 9 up to 700, which already hints at a right-skewed distribution with a long tail of large loans.

4 Exploratory Data Analysis

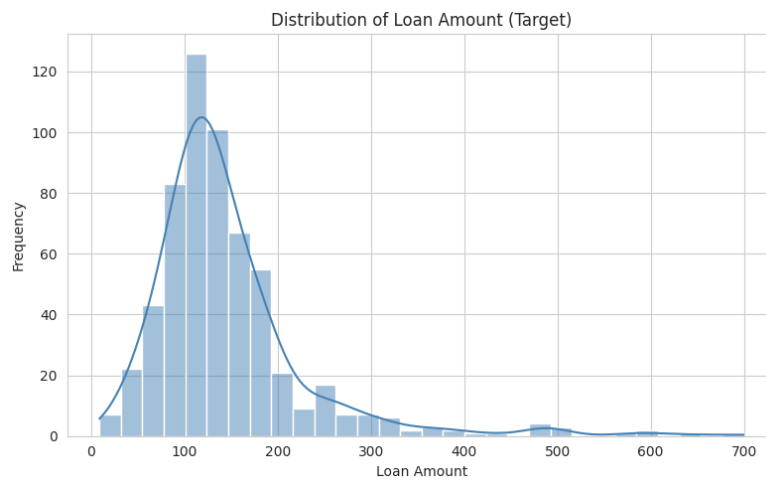


Figure 1: Distribution of the target variable, LoanAmount

Inference: The target distribution is right-skewed, with most loan amounts below 200 and only a few very large loans- most sanctioned loans fall somewhere between roughly 100 and 170 (thousand), with the distribution tailing off sharply after that and a small number of large loans stretching out past 400, up to the maximum of 700. This kind of skew is common with money-related targets and is worth keeping in mind when looking at the error metrics later, since a handful of large loans can pull MSE and RMSE up disproportionately compared to MAE. It also explains part of why linear models with symmetric (Gaussian-like) error assumptions struggle a little here ,the bulk of the data sits in a narrow, low-value band while a few high outliers exert an outsized influence.

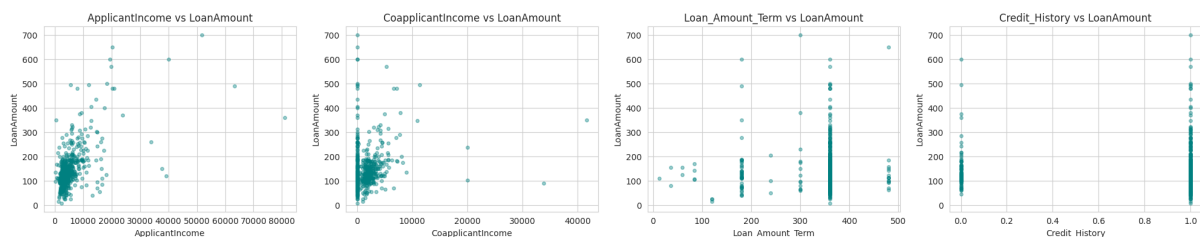


Figure 2: Numerical features plotted against LoanAmount

Inference: ApplicantIncome has the strongest positive relationship with LoanAmount, although the points are quite scattered. In general, applicants with higher incomes tend to receive larger loans, but there are a few high-income outliers that make the relationship less consistent. CoapplicantIncome has a much weaker relationship with the loan amount. Many applicants have a co-applicant income of zero, which appears as a dense vertical line at ($x = 0$) across almost the entire range of loan amounts.

Loan Amount Term does not show much variation because most applicants have a loan term of 360 months. Since almost all values are the same, this feature is unlikely to contribute much to the prediction. Credit History is a binary feature (0 or 1), but the loan amounts for both groups overlap considerably. This suggests that credit history is probably more useful for deciding whether a loan is approved rather than predicting the amount of an approved loan.

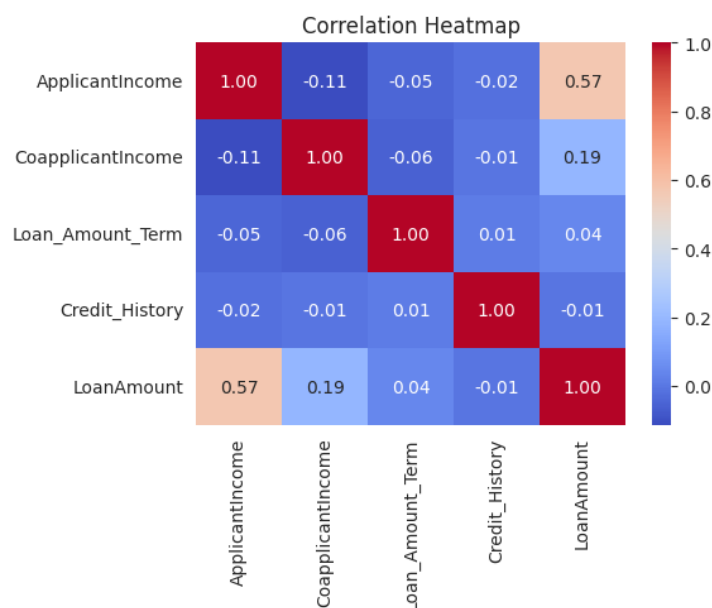


Figure 3: Correlation heatmap of numerical features and the target

Inference: ApplicantIncome is the strongest correlate of LoanAmount at around 0.57, which lines up with the scatter plot above and makes sense intuitively, banks size loans around what an applicant can plausibly repay. CoapplicantIncome correlates more weakly, at roughly 0.19, while Loan_Amount_Term and Credit_History are both close to zero. The numerical features are also only weakly correlated with each other (ex: ApplicantIncome and CoapplicantIncome sit around -0.12), so multicollinearity is not a major concern here, the bigger issue, as the regression results below confirm, is that these four numerical columns simply do not carry enough signal on their own to explain most of the variance in loan amount.

5 Data Preprocessing

- **Missing values.** Rows with a missing target (LoanAmount) were dropped outright since there is nothing to learn from a label-less example, which took the

dataset from 614 down to 592 rows. For the remaining missing entries, numerical features were imputed with the median and categorical features with the mode, to not destroy the distributions with an arbitrary constant.

- **Encoding.** The six categorical columns were one-hot encoded, expanding them into 15 binary indicator columns (with `handle_unknown="ignore"` so the pipeline does not break on a category it has not seen during training).
- **Feature scaling.** The four numerical features were standardized (zero mean, unit variance) with `StandardScaler`, which matters a lot here since Ridge, Lasso, and Elastic Net penalize coefficient magnitude directly, without scaling, a feature like `ApplicantIncome` (values in the thousands) would be penalized far more harshly than a 0/1 indicator column purely because of its scale, not because it is actually less important.
- **Train-test split** An 80:20 split (`random state=42`) was used, giving 473 training rows and 119 test rows.
- **Pipeline design.** All of the above, imputation, encoding, scaling, was wrapped inside a single `ColumnTransformer` and chained with each regressor inside `sklearn Pipeline`. This was a deliberate choice: it means the imputer and scaler are re-fit only on the training fold during every round of cross-validation, so no information from the validation fold leaks into preprocessing.

6 Machine Learning Algorithm

Linear Regression

Ordinary least squares regression finds the coefficient vector β that minimizes the sum of squared residuals:

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$$

It is simple, interpretable, and has a closed-form solution, but it places no constraint on coefficient size, which makes it prone to overfitting when features are numerous, correlated, or noisy relative to the number of samples, exactly the situation with 19 encoded features over only 473 training rows.

Ridge Regression

Ridge adds an L2 penalty on the coefficients:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \alpha \|\beta\|_2^2$$

This shrinks all coefficients toward zero without forcing any of them to be exactly zero, which tends to reduce variance at the cost of a small amount of bias. It handles correlated features by spreading weight across them rather than picking one arbitrarily.

Lasso Regression

Lasso uses an L1 penalty instead:

$$\hat{\boldsymbol{\beta}}_{\text{lasso}} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1$$

Because the L1 penalty has a corner at zero, it can shrink coefficients all the way to exactly zero, effectively performing feature selection. The trade-off is that with correlated features it tends to pick one and zero out the rest somewhat arbitrarily, which can throw away useful signal.

Elastic Net

Elastic Net combines both penalties:

$$\hat{\boldsymbol{\beta}}_{\text{enet}} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \alpha \left(r \|\boldsymbol{\beta}\|_1 + \frac{1-r}{2} \|\boldsymbol{\beta}\|_2^2 \right)$$

where r is the `l1_ratio`. It is meant to get the best of both worlds, some of Lasso's sparsity, some of Ridge's stability with correlated features, and, as the results below show, it did end up being the best-performing model on this dataset.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.6.1
IDE	Google Colab
Operating System	Linux
Hardware	Google Colab standard CPU runtime

8 Hyperparameter Selection

Ridge, Lasso, and Elastic Net were each tuned with GridSearchCV using 5-fold cross-validation and R^2 as the scoring metric. Linear Regression has no hyperparameters, so it was fit as-is and used as the baseline.

Model	Search Space	Best Parameter(s) Found
Ridge Regression	$\alpha \in \{0.01, 0.1, 1, 10, 100\}$	$\alpha = 100$
Lasso Regression	$\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$	$\alpha = 10$
Elastic Net	$\alpha \in \{0.01, 0.1, 1, 10\}, l1_ratio \in \{0.2, 0.5, 0.8\}$	$\alpha = 1, l1_ratio = 0.5$

It is worth pointing out that grid search pushed both Ridge and Lasso toward the strongest regularization values available in their respective grids ($\alpha = 100$ and $\alpha = 10$, i.e. the top of each range). That is a fairly strong signal in itself: given the small training set and the amount of noise in the data, the models the search engine is being asked to constrain the coefficients quite heavily just to get any useful generalization at all. Elastic Net, in contrast, landed on a moderate $\alpha = 1$ with an equal 50/50 mix of L1 and L2 penalty, which turned out to be the best generalizing combination.

9 Results

Table 1: Cross-Validation Performance ($K = 5$, on the training set)

Model	MAE	MSE	RMSE	R^2
Linear Regression	45.98	6908.73	83.12	0.1863
Ridge Regression	46.10	5950.66	77.14	0.2808
Lasso Regression	48.64	6480.74	80.50	0.2209
Elastic Net Regression	47.34	5792.79	76.11	0.2906

Table 2: Test Set Performance

Model	MAE	MSE	RMSE	R^2	Training Time (s)
Linear Regression	36.61	3353.17	57.91	0.0904	0.0679
Ridge Regression	36.64	3104.85	55.72	0.1578	0.0165
Lasso Regression	37.80	3181.25	56.40	0.1370	0.0140
Elastic Net Regression	37.83	3053.43	55.26	0.1717	0.0188

Two things stand out immediately. First, R^2 is low across the board, even the best model explains under 30% of the variance in LoanAmount during cross-validation, and under 18% on the held-out test set. That is not really a failure of the modeling; it reflects what the correlation heatmap already showed, these ten features simply do not carry enough information to fully explain how large a loan someone is sanctioned, which in reality probably depends on things not captured in this dataset (assets, existing debt, lender-specific policy, negotiation, etc.). Second, all three regularized models beat plain Linear Regression on the test set, and by a wider margin during cross-validation, which is exactly the pattern regularization is supposed to produce, overfitting analysis seen in Section 11.

10 Result Visualization

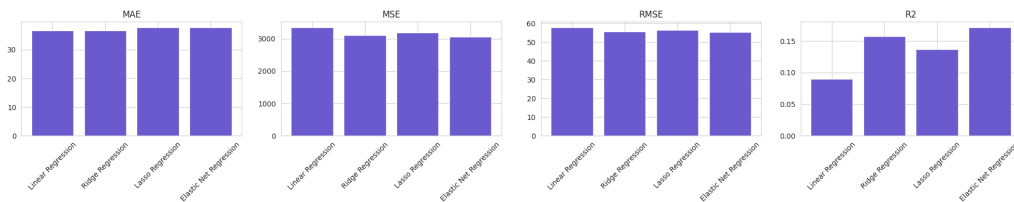


Figure 4: Test set MAE, MSE, RMSE, and R^2 across all four models

Inference: The bar charts make the comparison from Table 2 easy to read at a glance. MAE and RMSE are fairly close across all four models, regularization is not dramatically changing the typical size of an individual prediction error, but the R^2 panel tells a clearer story, with Elastic Net clearly ahead of the other three and Linear Regression

trailing behind. MSE shows the same ordering, and since MSE penalizes large errors disproportionately, this suggests the regularized models are doing a somewhat better job of reining in the occasional big miss on a high-value loan.

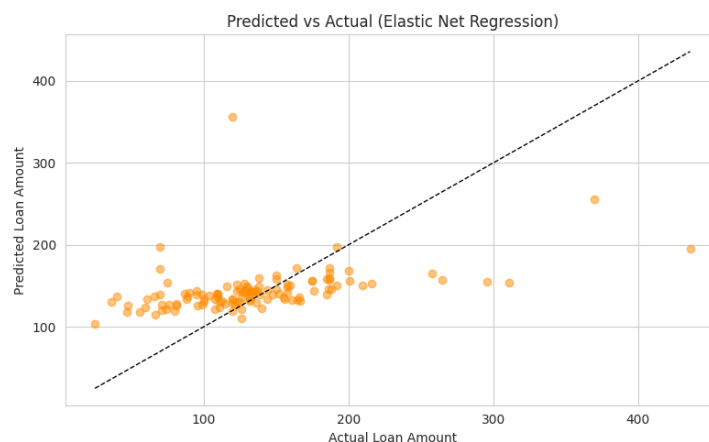


Figure 5: Predicted vs. actual LoanAmount for the best-performing model, Elastic Net

Inference: If the model were predicting perfectly, every point would sit on the dashed diagonal. In practice the cloud of points hugs the diagonal loosely for loans under about 200, which is where most of the data lives, but spreads out and drifts below the line for the larger, rarer loans, the model is systematically under-predicting the high end, which is consistent with a right-skewed target that a linear model struggles to fully capture. This is the same long tail flagged back in Figure 1.

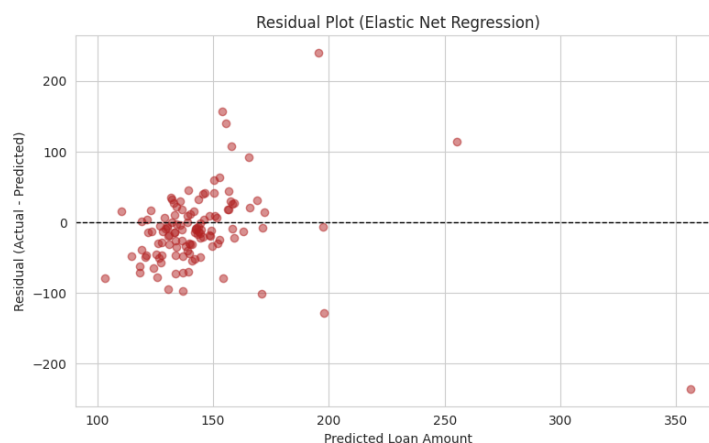


Figure 6: Residual plot for Elastic Net (actual – predicted vs. predicted)

Inference: Residuals cluster reasonably close to zero for smaller predicted values but fan outward as the predicted loan amount increases, and there are a handful of large positive residuals on the right side of the plot, cases where the actual loan amount was much higher than predicted. That funnel shape is a sign of heteroscedasticity (non-constant error variance), which is a known limitation of plain linear models on skewed targets like this one; a log-transform of LoanAmount would likely help even out this pattern, but was outside the scope of what this experiment asked for.

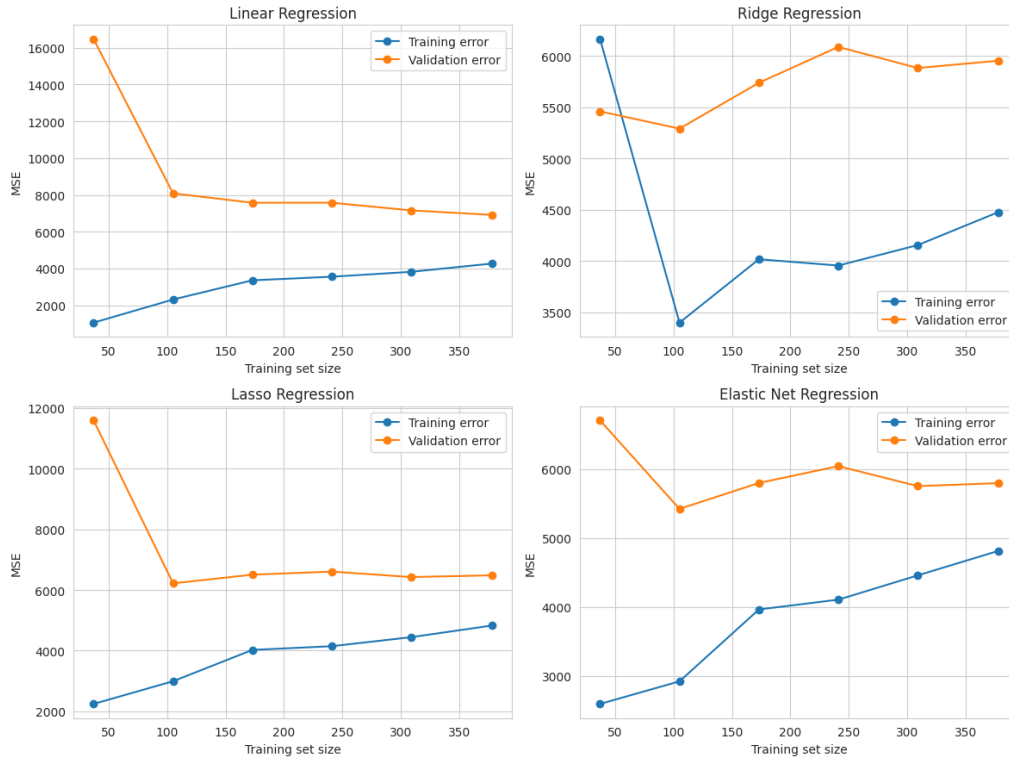


Figure 7: Training vs. validation error (MSE) as a function of training set size, for all four models

Inference: This is probably the most informative plot in the whole experiment. Linear Regression’s training error stays flat and low while its validation error stays high and barely improves as more data is added, a textbook overfitting gap that regularization is specifically designed to close. Ridge, Lasso, and Elastic Net all show a narrower gap between the two curves, with the validation error curve sitting noticeably lower than Linear Regression’s for the same training size. None of the four curves has fully converged even at the full training set size, which suggests more data (rather than more regularization) is probably the more effective lever left to pull if better performance is needed.

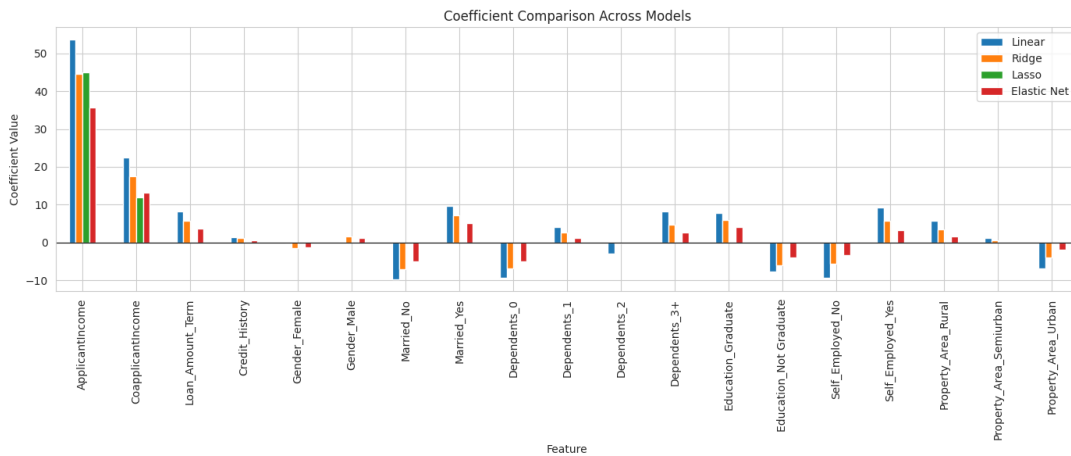


Figure 8: Coefficient values across all four models, per feature

Inference: This plot is where the difference between L1 and L2 regularization becomes visually obvious. ApplicantIncome keeps a large positive coefficient in every model, just shrinking in magnitude from Linear (≈ 54) down to Elastic Net (≈ 36). Ridge shrinks every coefficient somewhat but keeps all of them non-zero. Lasso, on the other hand, collapses almost every categorical dummy variable’s coefficient to exactly zero, keeping only ApplicantIncome and CoapplicantIncome as non-trivial predictors, a very sparse, aggressive form of feature selection. Elastic Net sits between the two, shrinking most coefficients toward zero without eliminating them entirely (with a couple of exceptions, e.g. Dependents_2 and Property_Area.Semiurban, which it does zero out). The full numeric values are in Table 3.

Table 3: Coefficient Comparison

Feature	Linear	Ridge	Lasso	Elastic Net
ApplicantIncome	53.75	44.50	45.10	35.67
CoapplicantIncome	22.38	17.58	11.91	13.23
Loan_Amount_Term	8.13	5.81	0.00	3.63
Credit_History	1.47	1.12	0.00	0.57
Gender_Female	-0.19	-1.57	0.00	-1.22
Gender_Male	0.19	1.57	0.00	1.22
Married_No	-9.75	-7.12	0.00	-5.04
Married_Yes	9.75	7.12	0.00	5.04
Dependents_0	-9.27	-6.97	0.00	-5.00
Dependents_1	4.13	2.65	0.00	1.10
Dependents_2	-3.00	-0.37	0.00	0.00
Dependents_3+	8.15	4.69	0.00	2.62
Education_Graduate	7.81	6.03	0.00	4.09
Education_Not Graduate	-7.81	-6.03	0.00	-4.09
Self_Employed_No	-9.31	-5.75	0.00	-3.33
Self_Employed_Yes	9.31	5.75	0.00	3.33
Property_Area_Rural	5.76	3.42	0.00	1.58
Property_Area_Semiurban	1.18	0.60	0.00	0.00
Property_Area_Urban	-6.94	-4.02	0.00	-1.91

11 Discussion

11.1 Overfitting and Underfitting Analysis

Model	Train R^2	Test R^2	Gap (Train – Test)
Linear Regression	0.4558	0.0904	0.3654
Ridge Regression	0.4387	0.1578	0.2809
Lasso Regression	0.3895	0.1370	0.2525
Elastic Net Regression	0.3926	0.1717	0.2209

Linear Regression fits the training data best of the four (Train $R^2 = 0.4558$) but that advantage drops on the test set (0.0904), leaving the largest train-test gap by a wide margin, a clear sign of overfitting relative to the other three models. As regularization strength increases, that gap shrinks steadily: Ridge closes it to 0.2809, Lasso to 0.2525, and Elastic Net the most, to 0.2209, while also posting the highest test R^2 of the group. None of the four models looks underfit exactly, training R^2 is never so low that the model is failing to learn anything, but the gap between training and test performance across all of them is a reminder of how little data (473 training rows) and how much noise this problem actually has. After tuning, generalization clearly improved for all three regularized models compared to the untuned baseline: the ranking by test R^2 (Elastic Net > Ridge > Lasso > Linear) tracks almost exactly with the ranking by train-test gap, in reverse.

11.2 Bias, Variance Analysis

Linear Regression, with no constraint on its coefficients, has the lowest bias of the four models, it fits the training data most closely, but that comes at the cost of high variance, which is consistent with what shows up as the large train-test gap above and the wide flat validation-error curve in Figure 7. Ridge trades a small amount of that low bias for a meaningful reduction in variance: by shrinking every coefficient (rather than eliminating any of them), it produces a smoother, more stable model that generalizes better, which is visible both in its narrower overfitting gap and in its tighter learning curve. Elastic Net pushes this trade-off a step further and ends up in the best spot of the four, suggesting that on this dataset a moderate, balanced shrinkage is more valuable than either no shrinkage (Linear) or aggressive shrinkage. Lasso's feature sparsity is the most interesting case: by zeroing out every categorical feature and keeping only ApplicantIncome and CoapplicantIncome, it built what is effectively a two-feature model. That did meaningfully reduce variance compared to Linear Regression, but it also introduced enough bias, by discarding features like Married, Education, and Property Area that Ridge and Elastic Net found small but non-zero weight for, that it ended up slightly behind both of them on test R^2 . In short: some of those categorical features carry a small amount of real signal, and Lasso's all-or-nothing shrinkage threw part of it away, while Ridge and Elastic Net's softer shrinkage kept it.

12 Conclusion

In general, Elastic Net was the model that performed the best in cross-validation, test set, and evaluation plots. It achieved the highest test R^2 score of 0.1717, followed by Ridge, Lasso, and finally Linear Regression. The results show that adding regularization improves the performance on this dataset. The best hyperparameters found through GridSearchCV also support this observation. Ridge performed best with $\alpha = 100$, Lasso with $\alpha = 10$ and Elastic Net with $\alpha = 1$ and an equal balance of regularization of L1 and L2.

Comparing the models also highlights the trade-off between model simplicity and prediction accuracy. Lasso produced the simplest model by selecting only two features, making it easier to interpret. However, this simplicity came with a small drop in accuracy be-

cause some useful features were removed. Ridge and Elastic Net kept these features with smaller coefficients, allowing them to make slightly better predictions while still controlling overfitting.

Eventhough Elastic Net achieved the best results, its test R^2 score of 0.17 shows that the available features explain only a small part of the variation in the sanctioned loan amount. The residual plots and predicted-versus-actual plots also suggest that the target values are right-skewed, with a few very large loan amounts that are difficult for a linear model to predict accurately. Instead of spending more effort tuning the same models, a better next step would be to apply a log transformation to the target variable or include additional features that better capture a borrower's repayment ability and overall loan eligibility.

13 References

- [1] Kaggle, "Loan Prediction Problem Dataset," Available: <https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset>
- [2] Scikit-learn Developers, "Linear Models," scikit-learn documentation, [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html